A batch runs on a chain of command

> **AI Assistant Note**: Read this when deciding which model takes which role in a batch of parallel sessions, or when a rank is unsure whether a decision is theirs to make. Each decision is made at the lowest rank that can make it well, and reaches the user only when the general can't advise on it.

A batch holds more decisions than the user wants to make. A chain of command, each rank on the model that suits its judgement and its cost, takes cards that don't depend on each other from Linear to Done while the user hears from it only occasionally.

The principle: each rank works at its sweet spot, and only the user's decisions reach the user

The sweet spot is where more effort stops buying better work: below it you lose quality, above it you pay for nothing. The user's attention is the dearest resource in the chain, so a question climbs only as high as it has to.

Every rank's model and effort is chosen against the first half, and the escalation rule below carries out the second. The tests of the chain of command check both.

Quick reference

Rank	Role	Model	Effort	Passes a decision up when
Commander-in-chief	The user. Sets the mission: what each card is for and how far it goes	—	—	—
General	A senior officer	Fable	—	

Rank	Role	Model	Effort	Passes a decision up when
	who advises the captain when asked, through `/advisor`			The decision is about purpose or scope, or needs something only the user knows
Captain	Runs the batch: starts the soldiers, hears from them, checks each branch combined with `main`, sends work back, merges	Opus	High	It can't settle a decision itself
Soldier	Carries out one card in its own worktree and brings its questions to the captain, not the advisor	Sonnet	High	It hits an error it can't fix, or a question about the card's purpose or scope

Ranks are roles, not models: the model and effort columns are today's choice, and a soldier whose plan is worked out during the work runs on Opus. The flags that apply each setting are in the chain of command config.

The escalation rule

A soldier doesn't ring the general, and the general isn't consulted about the coffee. Each decision goes up one rank only when the rank below can't make it, so the user hears from a batch only occasionally; if they're asked often, a rank below is passing up what it could settle. The principle is mission command: give every rank the intent, and decide at the lowest rank that can.

The ranks

The soldier does focused work where the plan is known. Sonnet takes the rank because it implements well and costs half as much per token as Opus. Whether a card costs less overall depends on the effort, and high effort gives the best work for the least use; see the config. It carries the plan across as many files as it takes and makes the calls inside it, such as which errors a narrowed catch should still handle. When the plan is worked out during the work, as in moving code, Opus takes the rank. The soldier's brief sends its questions to the captain rather than the advisor, and a soldier started as a `claude -p` session has the advisor switched off as well. The captain is the right rank to answer them, because it knows what every other soldier is doing.

The captain is the only rank that sees across cards. It spots what one card's merge does to the next, such as one card's new lint rule breaking another card's new test helper. Most of its work is mechanical; the judgement is in sending work back and resuming soldiers, which is why it runs at high effort.

The general advises when asked and gives no orders. It reads the captain's whole conversation and gives a stronger second opinion, and the captain makes the decision, so the user isn't stopped. The captain calls it before any question goes to the user, once it has gathered what the soldiers reported, so the general sees the whole batch rather than one card. Fable takes the rank because a different model gives a different view: an Opus general would share the captain's blind spots.

The commander-in-chief decides what the campaign is for and leaves how to fight it to the ranks. Purpose and scope are the user's, so the general passes them up rather than answering them.

A snag that recurs goes up once, as a fix to the process. When the same problem turns up on job after job, the captain stops working around it and brings it up with a worked before and after; the fix lands in the work, the written record, and the tool that let it through. See learn from the run.

Inspectors check the work from outside the chain

The reviewers aren't a rank: they check the work against the card and report, and take no orders. They keep the models their workflows name whatever the soldier runs on, which is what makes a cheaper soldier safe.

The captain is the one rank that can't fail

The soldiers run as child processes of the captain's session, so they end when it ends. Their pushed work survives on their branches, `claude -p --resume` restarts them, and the batch's state lives in its progress file, `work/progress.md`, which a new captain can pick up.

A worked example: three cards from the board to Done

A captain on Opus, with Fable as general, runs three code-change cards that don't depend on each other on Sonnet soldiers, one worktree each. The first card merges and adds a lint rule, and the second card's new test helper breaks it, so the captain sends the second card back and its soldier fixes it on its branch. The third card's session has exited by the time its branch conflicts with `main`, so the captain resumes it with the conflict described. None of the card work needs the user.

What makes it work

Every soldier can reach the captain, the captain checks each branch combined with `main` before merging it, and the soldier that wrote the code fixes it, so the inspectors' record stays true. The brief says how; see BRIEF.md.